Chapter 6: Supervised Learning: Concepts and Definitions

Outline

6.1 Introduction

- There are two distinct types of problems in supervised learning (SL): regression (continuous outputs) and classification (discrete classes).
- Common algorithms: Support vector machines (SVM), decision trees (DT), k-nearest neighbors (k-NN), artificial neural networks (ANN), and ensembles.

6.2 Supervised Learning as a Search Problem

- SL can be understood as the task of *finding* a hypothesis that explain the behavior of a system in such a way that one is able to *predict* its behavior in an unseen scenario.
- The process of finding this hypothesis, which is contained in the model's parameters, is called *training*.
- To train a model we need:
  - Examples
  - Performance metric
  - Recursive method to automatically modify the hypothesis (model's parameters) to improve the performance.

6.2 Supervised Learning as a Search Problem - Continued

Example	Pressure	Temperature	Humidity	Set-point	Fault
1	High	High	Normal	Same	Yes
2	High	High	High	Same	Yes
3	Low	Low	High	Change	No
4	High	High	High	Change	Yes

Table: Example data for supervised learning.

- Consider a process with four variables (features) that can be classified as faulty or not.
- The goal is to find the best hypothesis, based on the data available, that predicts if the process will be faulty,

6.2 Supervised Learning as a Search Problem - Continued

- The most general hypothesis we can propose is $\langle ?, ?, ?, ? \rangle \rightarrow \textit{yes}$, which means that any feature can take any value and the output will be "yes".
- The general hypothesis contradicts example #3, so it is wrong.
- The most specific hypothesis is $\langle \emptyset, \emptyset, \emptyset, \emptyset \rangle \rightarrow \textit{yes}$, which means that there is no value an attribute can take for the output to be "yes".
- Since some examples are labeled as "yes", we know the specific hypothesis is also wrong.
- Given that both the most general and the most specific hypotheses are wrong, we can assume that the correct one lies somewhere in between.

6.2 Supervised Learning as a Search Problem - Continued

- Start with the most specific hypothesis and apply it to the first example. Since none of the $\emptyset$ in the current hypothesis is satisfied by this example, we replace it with the *next most general* hypothesis that fits the example, that is $\langle High, High, Normal, Same \rangle \rightarrow yes$.
- Next, apply our new hypothesis to the second example. According to our hypothesis, humidity should be *Normal* for the output to be *yes*, which is contradicted by the current example. We then move to the next most general hypothesis that still agrees with the first example and accepts the new one: $\langle High, High, ?, Same \rangle \rightarrow yes$.

6.2 Supervised Learning as a Search Problem - Continued

- The latest hypothesis also agrees wit the third example, so no changes are needed.
- From the fourth example we learn that the last attribute, set-point, could take either value for the fault to be present, so we modify the hypothesis to be $\langle High, High, Normal, ? \rangle \rightarrow yes$.

6.2 Supervised Learning as a Search Problem - Continued

- This example illustrates the intuition behind training in SL.
- However, this procedure would be ineffective for large datasets, a higher number of features, or for a regression problem.
- There are at least three improvements needed:
  - A way to automatically modify the current hypothesis to agree with new information.
  - A way to deal with contradictions in the data.
  - A way to reduce the number of steps or iterations needed to incorporate all the information available in the data.

Supervised learning techniques

6.3 Supervised Learning Techniques

- The intuition behind SL has been implemented in many forms over the years, all oriented to address the challenges outlined before, as well as issues related to specific applications.
- These forms are sometimes referred to as algorithms, methods, architectures, model types, and others. To make things simple, we will just call them techniques or methods.
- There is no one-size-fits-all method, and each has its own advantages and shortcomings depending on the application, the data availability, data type, and many others.

6.3.1. Decision Trees

- Decision trees (DTs) focus on reducing the number of steps needed to evaluate all the data.
- DTs look at all the attributes simultaneously, and selects the one that produces the cleanest cut.
- In the fault detection example, the attribute *Pressure* does such a thing. When *Pressure* is *High* all outputs are *yes*, when it is *Low* all outputs are *No*.
- Once the dataset is partitioned, we focus on each partition separately, and continue until we exhausted all the attributes.

Example	Pressure	Temperature	Humidity	Set-point	Fault
1	High	High	Normal	Same	Yes
2	High	High	High	Same	Yes
3	Low	Low	High	Change	No
4	High	High	High	Change	Yes

Table: Example data for supervised learning.

6.3.1. Decision Trees - Continued

- To formalize the notion of *cleanest cut* we start by defining the entropy of a sample, $Entropy(S)$. For a binary classification problem:

$$Entropy(S) = -p_+ \log_2(p_+) - p_- \log_2(p_-) \tag{1}$$

- Here, p_+ is the proportion of positive examples, and p_- is the proportion of negative examples.

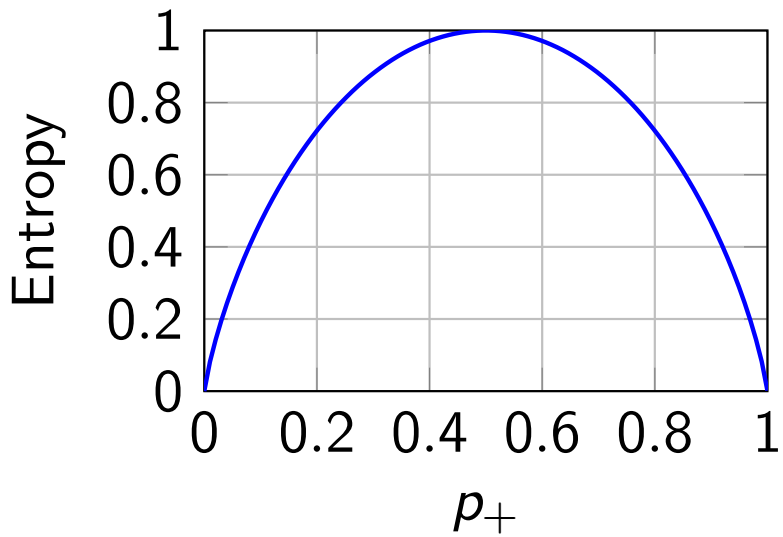

Figure: Entropy as a function of p_+ for a binary classification problem.

6.3.1. Decision Trees - Continued

- We now can select the best attribute to partition the data by choosing the one that *provides the greatest reduction in entropy*. This is called *information gain*
- The information gain in a sample S by selecting attribute A is:

$$Gain(S, A) = Entropy(S) - \sum_{v \in Values(A)} \frac{|S_v|}{|S|} Entropy(S_v) \qquad (2)$$

- Each point at which an attribute is used to partition is called a *node* and each partition is called a *branch*.
- The point where no more decisions are made is called a *leaf*.
- The final hypothesis is stored in the decisions made at each node.

6.3.1. Decision Trees - Continued

Algorithm 1: Decision Tree Algorithm

Input: Dataset D with attributes and outputs.

Output: Decision tree.

```
1  Select A, the attribute that best classifies the instances in D;
2  Use A to partition D into subsets and create a branch for each
     subset;
3  foreach branch do
4      if all instances in the subset have the same output then
5          Create a leaf and label it with the value of this output;
6      else
7          if all attributes have been used then
8              Create a leaf and label it with the majority output
                 value;
9          else
10             Recursively apply the algorithm to the subset;
```

Support Vector Machines

- DTs can be thrown off if some examples are mislabeled (i.e., data are noisy), as it would lead to and incorrect measurement of entropy and information gain.
- Support vector machines (SVMs) define the classification problem in a way that is inherently robust to noisy data.
- Instead of using one attribute at a time, SVMs try to find a *hyperplane* (across all dimensions) that separates the classes.
- Then, a hyperparameter can be tuned to defined how much crossover from one class to the other is allowed over the boundary.

Support Vector Machines - Continued

- In practice, SVMs find the *support vectors* at the boundary of each class that are also the farthest apart.
- The hypothesis we are searching for is represented by these support vectors.

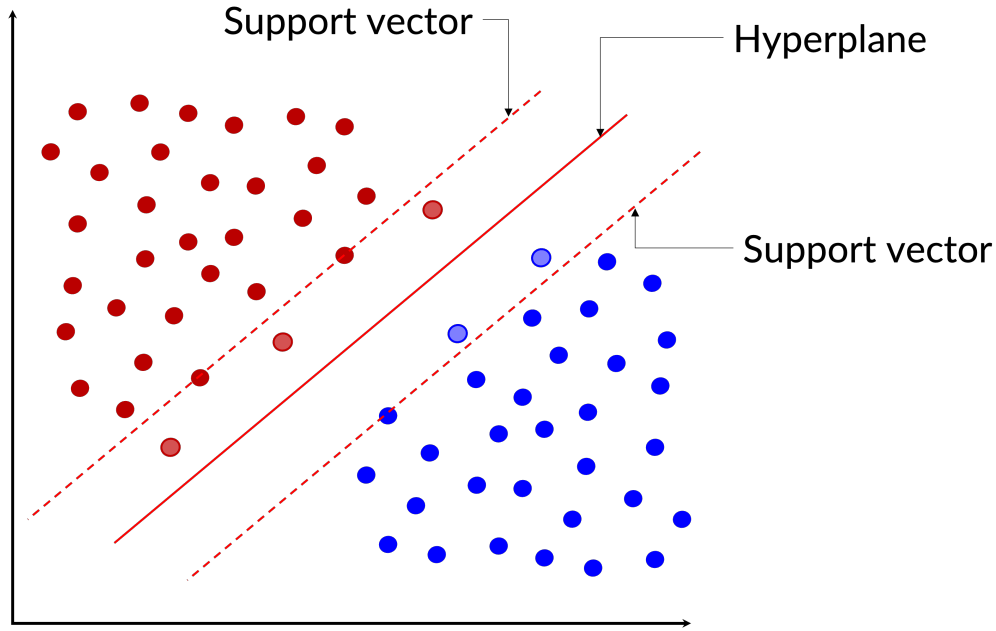

Figure: Support vector machines and hyperplane separation.

Support Vector Machines - Continued

- If data are not linearly separable, that is, not hyperplane can be trace between the classes with enough separation, a transformation known as *kernel function* is used to project the data into a new space in which it is linearly separable.

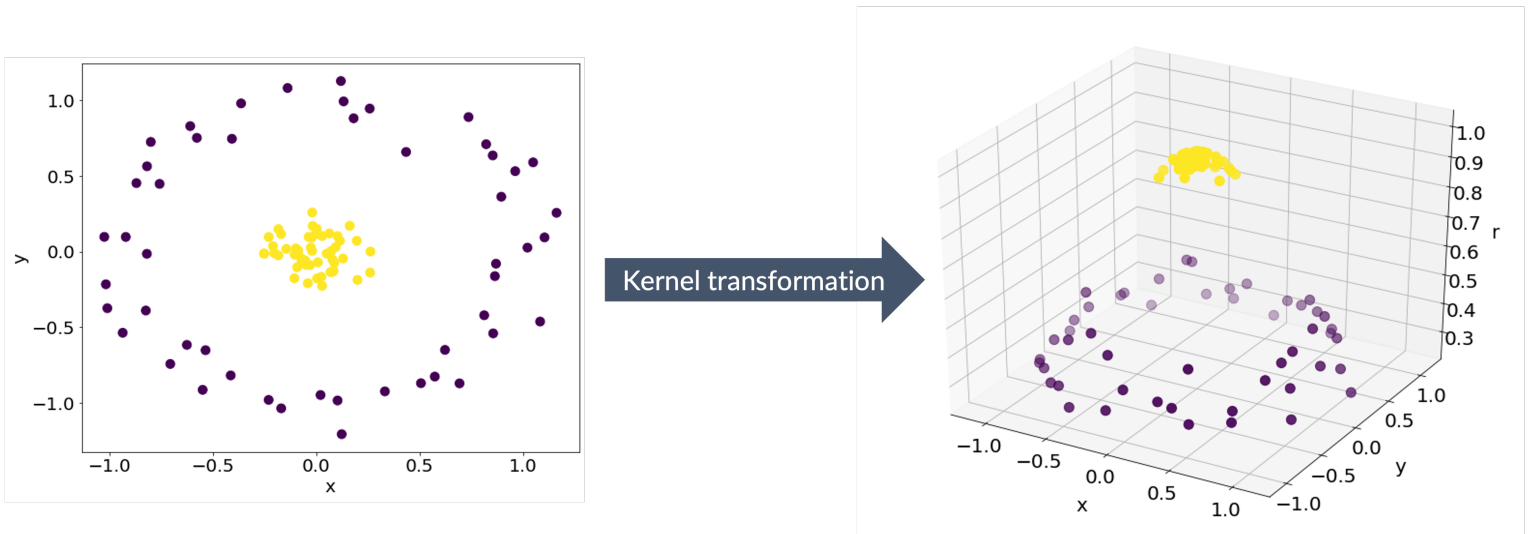

Figure: Kernel transformation.

Support Vector Machines - Continued

- Support vector regression (SVR) uses the same principle as SVMs but instead of finding the hyperplane that better separates the data, it finds the support vectors that are closer to each other that contain most of the data.

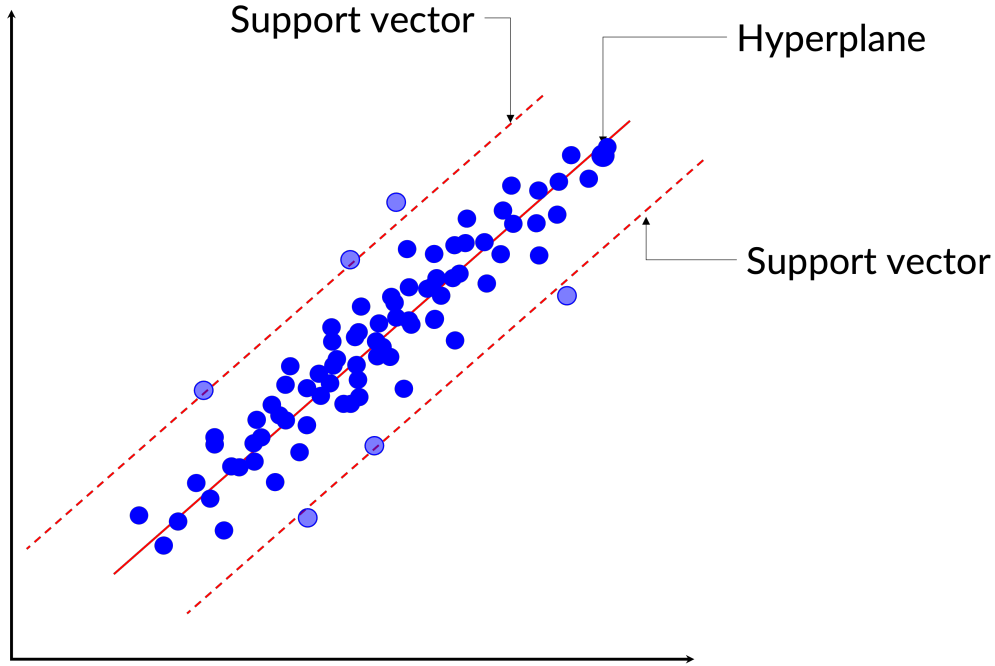

Figure: Support vector regression.

Adaptive k-Nearest Neighbors (Ak-NN)

- If the boundary between classes is highly non-linear, a more flexible approach might be needed.
- In k-NN, the boundary is based on the distance between the members of a class.

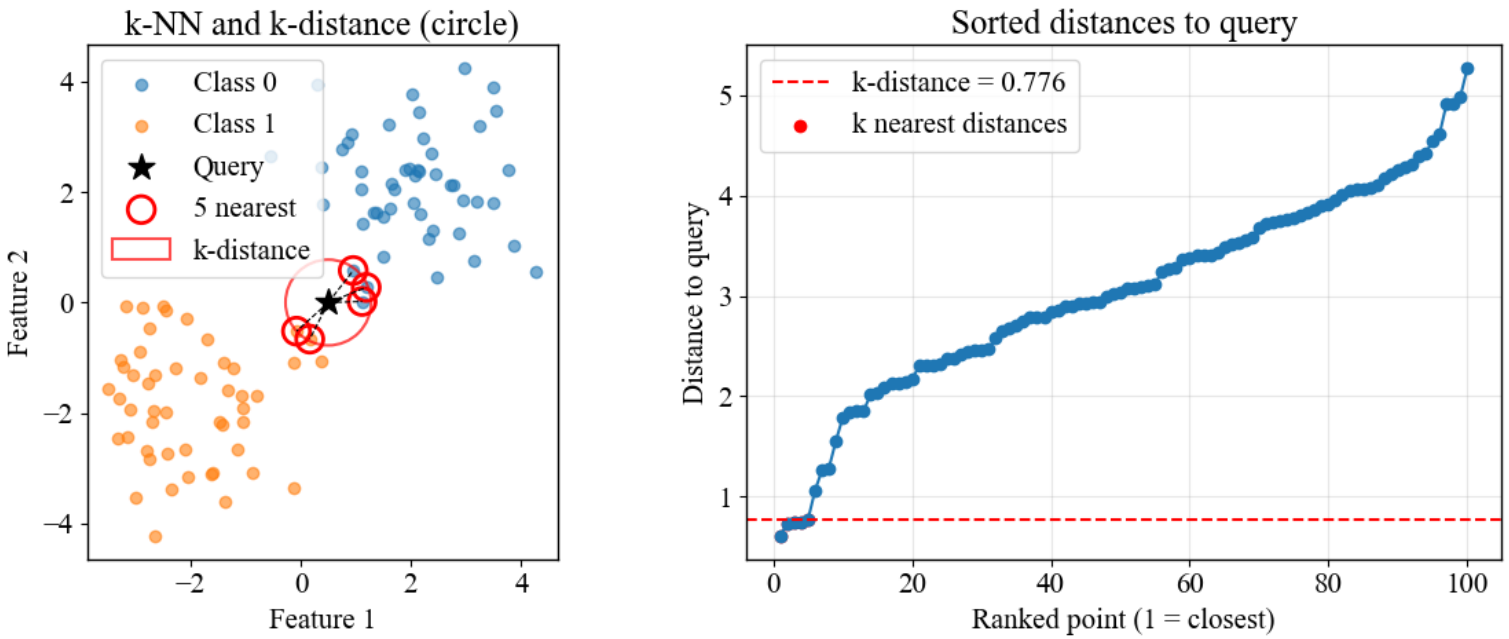

Figure: k-nearest neighbors classification.

Adaptive k-Nearest Neighbors (Ak-NN) - Continued

- Adaptive k-NN uses the k-NN principle for fault detection in industrial applications.
- First, a clustering method is used to find the right distance threshold.
- A new sample is labeled as fault if it is beyond the threshold from a known normal operating state.
- If enough new information is available, the set of known normal operating states is updated.
- The hypothesis is then stored as the distance thresholds to assign a point to each cluster.

About Hypotheses Searching

- All the methods described before share an important characteristic: the complexity of the hypotheses we can explore is limited by the data we use for training.
- In other words, the size of the hypothesis set $\langle \textit{feature } 1, \textit{feature } 2, \ldots, \textit{feature } N \rangle$ is determined by the number features we have in the data.
- SVMs can expand this set using the kernel transformation, but the kernel itself is fixed and not trainable, which means we need to find the right kernel first.
- This characteristic can be an advantage as we will see later, but could also limit our ability to find the right hypothesis.

Artificial Neural Networks

- What if we could create an *arbitrarily complex hypothesis* and then use data to refine it?
- Artificial neural networks (ANNs) accomplish this by representing the hypothesis (the model) as nodes connected by edges and arranged in layers. We then can have as many nodes and as many layers as we want.
- The refinement of the hypothesis is done by evaluating the error in the predictions and (back)propagating it through the network.
- An important caveat of this is that we may not have enough data to adequately refined an initial hypothesis that is too complex. The balance between model complexity and data availability should always be considered.

Artificial Neural Networks

- The basic unit (node) of an ANN is called *perceptron*.
- The output of a perceptron is:

$$y = activation\left(\sum_{i=0}^{n} w_i x_i + b\right) \tag{3}$$

- Here, x_i are the inputs, w_i are the weights, b is the bias, and f is the activation function.
- The weights and bias store the model, that is, a hypothesis.
- *activation* is some function that transforms the output of the perceptron.

Artificial Neural Networks - Perceptron

- In an ANN, multiple perceptrons are combined in layers of arbitrary size. The choice of number of nodes and layers is sometimes called *architecture* of the ANN.

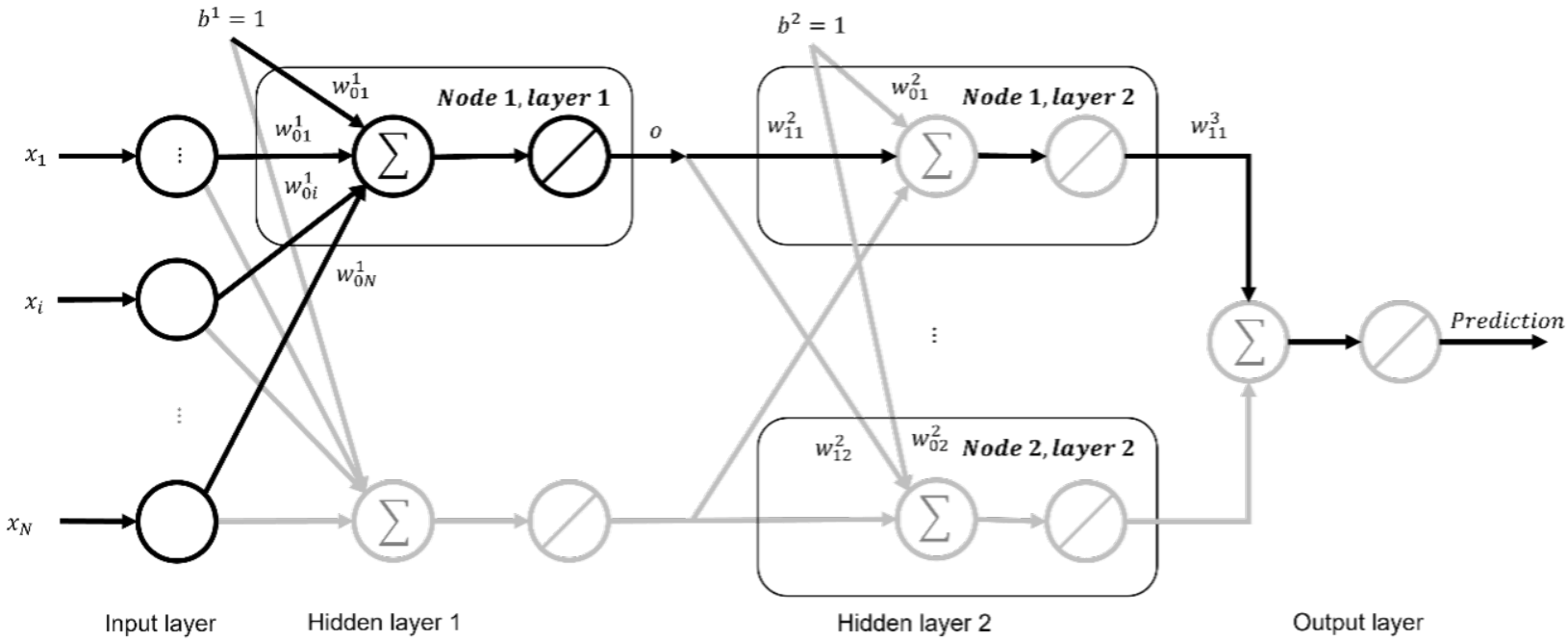

Figure: Artificial Neural Network architecture.

Artificial Neural Networks - Backpropagation

- ANNs are trained by calculating the predicted output for a given set of inputs, computing the error E between the result and the true value, and adjust the weights accordingly.
- The weights between the nodes i and j, w_{ij} us updated using the gradient descent algorithm:

$$w_{ij}^{updated} = w_{ij} - lr\frac{\partial E}{\partial w_{ij}} \tag{4}$$

- Here, lr is the learning rate, which regulates how much the weights change after each iteration.
- $\frac{\partial E}{\partial w_{ij}}$ is the partial derivative of the error with respect to each weight w_{ij}.

Artificial Neural Networks - Vanishing Gradient

- If the network has too many layers, the effect of the error in the last layers is almost negligible in the earlier layers.
- This phenomenon is known as *vanishing gradient*
- This problem can be magnified by the choice of activation function. *sigmoid* and *tanh* are known to induce gradient vanishing problems for deeper networks.
- Other activation functions were specifically designed to minimize the problem, such as *ReLU*.
- Other strategies include modifying the architecture of the ANN to bypass some error information to the earlier layers.

Ensemble Methods

- A useful approach to overcome the limitations of individual ML methods is to train two or more models on the same data and then obtain the final prediction from.
- Random Forests (RFs) are a common ensemble method, where hundreds of decision trees (DTs) are trained, and their outputs are averaged to improve resilience to noise and reduce overfitting.
- Ensembles can mix different ML methods (e.g., ANNs, DTs, SVMs) to exploit their respective advantages and measure prediction uncertainty, which is valuable in chemical engineering applications.
- Downsides include increased model complexity, a higher number of parameters, and difficulty in interpreting ensemble methods compared to individual models.

6.4 Hyperparameters

- Hyperparameters can be thought of as specifications or constraints in the the way the ML method searches for a valid hypothesis.
- In ANNs, the gradient descent algorithm finds the *best fitting* set of weights $W = \{w^0, \ldots, w^l, \ldots, w^L\}$, where $w^l \in \mathbb{R}^{n_i \times n_{i+1}}$, L is the number of layers in the ANN, and n_i is the number of nodes in the i—-th layer.
- There are infinite possible combinations for W, but the size and shape of the array (the number of weights to adjust) is fixed.
- The algorithm is then constraint to find the *best attainable solution* with this *number of weights*.
- It may happen that any *useful* solution lies outside this search space.

6.5 Overfitting and Regularization

- Overfitting is the problem of model adjusting too tightly to the training data so that it cannot make good predictions for new data.
- In our previous example, without the last record our hypothesis would have been $\langle High, High, ?, Same \rangle \rightarrow yes$ which would make a wrong prediction for row number 4.
- Without using new data for training, we could relax the hypothesis by turning each attribute to ? one at the time and test the model. The attribute for the which the accuracy drops the most is responsible for overfitting and should be kept as ?.

Example	Pressure	Temperature	Humidity	Set-point	Fault
1	High	High	Normal	Same	Yes
2	High	High	High	Same	Yes
3	Low	Low	High	Change	No
4	High	High	High	Change	Yes

Table: Example data for supervised learning.

6.5 Overfitting and Regularization

- The previous scenario illustrates how overfitting can be understood in two different ways:
  - Data availability
  - Model complexity
- In most applications, is safe to assume that the data we have is all the data available.
- This means that we should focus of model complexity to address overfitting.
- *Regularization* is the process of reducing model complexity.

6.5 Overfitting and Regularization - DT Pruning

- Model complexity in DTs can be described as the size of the tree, that is, the number of branches (decision nodes) and its depth (levels between root node and leaves).
- Pruning is a strategy in which a subtree rooted in a decision node is removed and replaced by a leaf, effectively reducing the model complexity.
- Pruning can be applied as the three is growing (during training) or after a completed three is obtained. The latter is more common.
- This can be applied iteratively until certain accuracy or model complexity is achieved.

6.5 Overfitting and Regularization - C and ϵ in SVR

- In SVR, ϵ can be understood as the allowed distance between the support vectors that contain the data.
- A large enough ϵ will make all points fall withing the region delimited by the support vectors, but make the regression model useless.
- A very small ϵ will force the hyperplane to pass through every single point, which would result in an overly complex model that will likely overfit.
- C controls how much attention is given to the points that fall beyond the support vectors.
- A larger C will penalize such points heavily, again forcing the hyperplane to bend to include them.
- A smaller C will be more forgiving, inducing less complexity in the model.

6.5 Overfitting and Regularization - Dropout

- In ANNs, dropout reduces the model complexity by setting some weights to zero, effectively removing them from the model.
- In practice, the weights set to zero are selected randomly, and the probabilty of a weight being zeroed is a hyperparameter typically set by the user per layer.
- In general, input layers are given a lower dropout probability, while a common value of 0.5 is used for the middle and output layers.

6.6 Method Selection

- The effectiveness of an ML method depends on the application and the data characteristics, such as dimensionality, size, diversity, and noise.
- DTs may struggle with noisy data due to their greedy nature, while ANNs are more resilient as noise is distributed through the model via backpropagation.
- High-dimensional data may lead to overfitting in ANNs without sufficient training data, whereas DTs handle high-dimensional data better due to their partitioning approach.
- Similarly to hyperparameter tuning, method selection is a search problem.

6.7 Feature Selection

- All features or attributes in a dataset might not be necessary for good model performance, and some might even hurt the model. Also, the necessary attributes to obtain a good model might not be readily available.
- So, before moving to selecting and optimizing an ML model, it is good practice to explore the data and get a feel of the effect of the attributes over the output variable.
- As a general rule, fewer attributes lead to simpler models.
- Correlation is a good first approach:
  - Input variables that have low correlation with the output can be discarded.
  - If two or more input variables are highly correlated to each other, domain knowledge should be used to select one to keep and discard the rest.

6.8 Feature Engineering

- Feature engineering is the process of transforming existing features or attributes before they are given as inputs to the ML model.
- This may be needed when the model is expecting a certain data type, like real numbers, and the feature is in a different data type, such as graphs.
- Feature engineering can also be used to infused domain knowledge in the model, by augmenting the data sets with additional information obtained from mechanistic models.

6.9 Transfer Learning

- In transfer learning (TL), a data-driven model previously trained (general training) for a given task (source domain) is used as the base to build a model for a new task (target domain), with fewer data being required for the new training stage (task-specific training).
- Given a source domain $\mathcal{D}_S$, a learning task $\mathcal{T}_S$, a target domain $\mathcal{D}_T$, and a learning task $\mathcal{T}_T$, transfer learning aims to help the learning of the target predictive function $f_T(\cdot)$ for the target domain using the knowledge in $\mathcal{D}_S$ and $\mathcal{T}_S$ where $\mathcal{D}_S \neq \mathcal{D}_T$ and $\mathcal{T}_S \neq \mathcal{T}_T$.
- TL allows the use of highly complex models in applications with scarce availability of data.

6.10 Conclusions

- Building a supervised learning model involves finding the best hypothesis that explains the data available.
- Dimensionality (number of features) and dataset size (number of examples), are crucial to balance model complexity and data availability to prevent overfitting.
- Model architecture and hyperparameters play a key role in guiding and constraining the search for a good hypothesis.
  - Complex architectures have large combinatorial spaces for hyperparameters and require extensive data to achieve good performance.
  - Simpler architectures are easier to tune and require smaller datasets but may not achieve satisfactory performance.
- Regularization, hyperparameter tuning, feature selection, feature engineering, and transfer learning all are ways to strike a balance between model complexity and data requirements.